

Aviator Design Document

David Thoe, Joshua Kim, Zeke Ulrich, Juan Vargas

December 13, 2025

GTA: Ruihang Ni
Professor: Minghao Qi

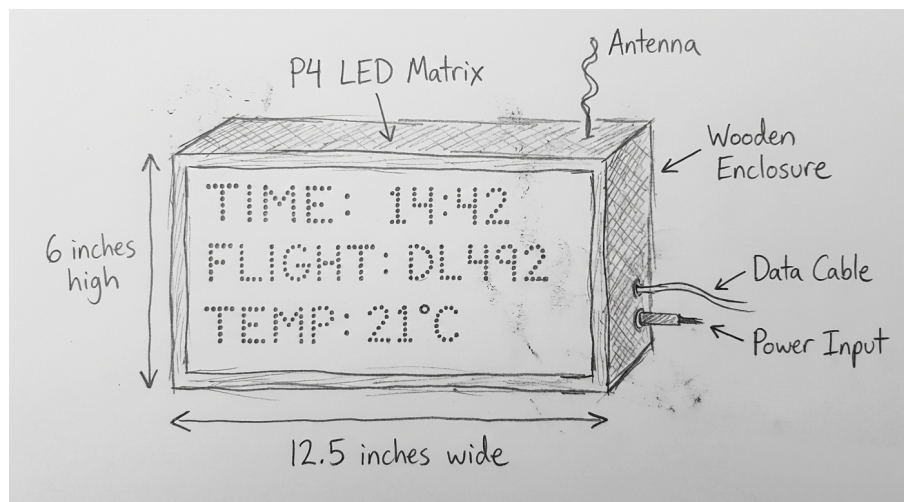


Figure 0.0.1: Sketch

Contents

List of Figures

List of Tables

1 Revision Log

Date	Revision	Changes	Initials
05/03/2024	v0.1	Initial Release	ZU
09/18/2025	v0.2	First Submission	ZU
10/05/2025	v0.3	Second Submission	DT
11/11/2025	v0.4	Corrections, Updates	DT
12/13/2025	v1.0	Final Release	ZU

Table 1: Revision Log

2 Glossary

- **API** Application Programming Interface.
- **ADS-B** Automatic Dependent Surveillance-Broadcast

3 Introduction

3.1 Executive Description

Retro-themed aircraft information display for hobbyists and novices.

3.2 User Stories

The Long-Time Aviation Hobbyist

As a long-time aviation hobbyist who has spent years tracking flights through phone apps, I'm tired of paying for subscriptions just to unlock basic features. I want a device that gives me real-time flight information without hidden costs, while also providing a tactile, nostalgic experience that reminds me of classic aviation boards. By having the Aviator on my desk, I can finally stay connected to the aviation world without feeling like I'm paying a premium for something that should be standard and accessible.

The Casual Aviation Enthusiast

As someone with a general interest in aviation, I don't need a full cockpit-level tracker, but I do want something that feels engaging and easy to use. Standard apps are flat, cluttered, and frankly too much for a novice like myself, but the Aviator project makes flight tracking simple, physical, and fun. I can glance at the board, see arrivals and departures, and feel connected to the aviation scene effortlessly. The setup process was simply plug and play. Additionally, I don't have to pay a dime for the product. For me, it's about accessibility and enjoying aviation in a personal, low-effort, high-impact way.

The Purdue ECE Student

As a Purdue ECE student, I'm drawn to the Aviator not only as a hobby project that I can tinker with, but also as a nod to Purdue's deep aviation legacy. It's inspiring to own a piece of tech that bridges my academic interests in circuits and embedded systems with Purdue's reputation in aerospace. I want a tracker that feels hands-on, customizable, and personal—something that makes me feel part of both my field of study and Purdue's aviation history every time I glance at it. Given the nature of the project and its ability to be completed by an individual excites me, as it gives me the stepping stone I needed to start tracking flights.

4 Design Requirements

4.1 Requirements

1. The device must display accurate information.
2. The display must not interfere with user's well-being by, for example, displaying at excessive luminosity, updating rapidly in a distracting manner, or being excessively bulky.
3. The device must not infringe on any person's reasonable expectation of privacy.
4. The device must be language-agnostic wherever possible.
5. The device must be responsive and intuitive.
6. The device must have robust error handling and recovery.
7. The physical device should be easily replicated with widely available parts.
8. The code for the device must be open-source and well-documented.
9. The device should be as durable and environmentally friendly as possible so as not to contribute to e-waste.
10. The device must not contribute to noise or visual pollution of any space.
11. The device must be energy-efficient.
12. The device must minimize construction and recurring costs.
13. The device must not infringe on right to repair.
14. The device must mount and dismount without damage to vertical surfaces.

4.2 Factors Influencing Requirements

4.2.1 Public Health, Safety, and Welfare

1. User well-being
2. Privacy

4.2.2 Cultural Factors

1. Language differences
2. Ease of use

4.2.3 Social Factors

1. Ease of replication
2. Open-source and documentation

4.2.4 Environmental Factors

1. Environmental friendliness and e-waste
2. Noise and visual pollution
3. Energy efficiency

4.2.5 Economic Factors

1. Cost
2. Repairability

5 System Overview

5.1 System Block Diagram

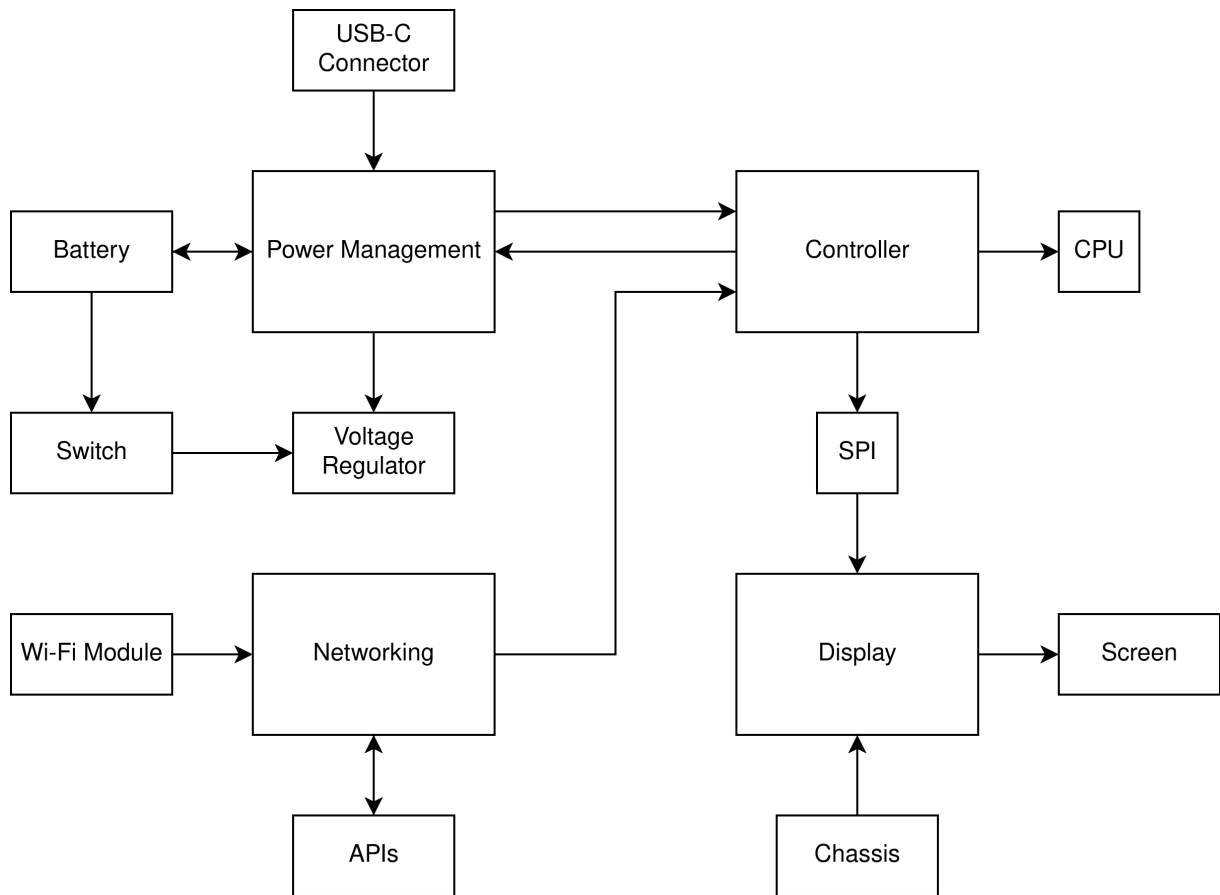


Figure 5.1.1: Aviator Block Diagram

5.2 System Activity Diagram

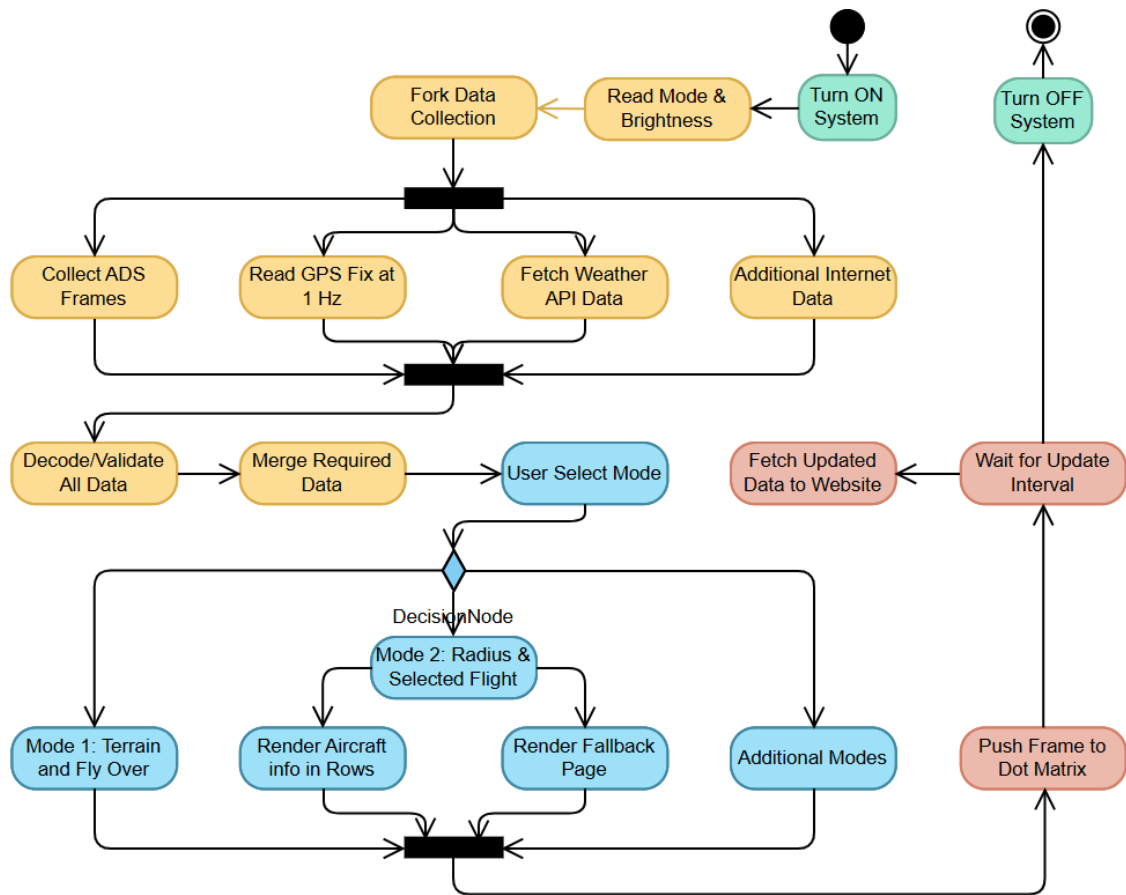


Figure 5.2.1: Aviator Activity Diagram

5.3 System Mechanical Design (Extra Credit)

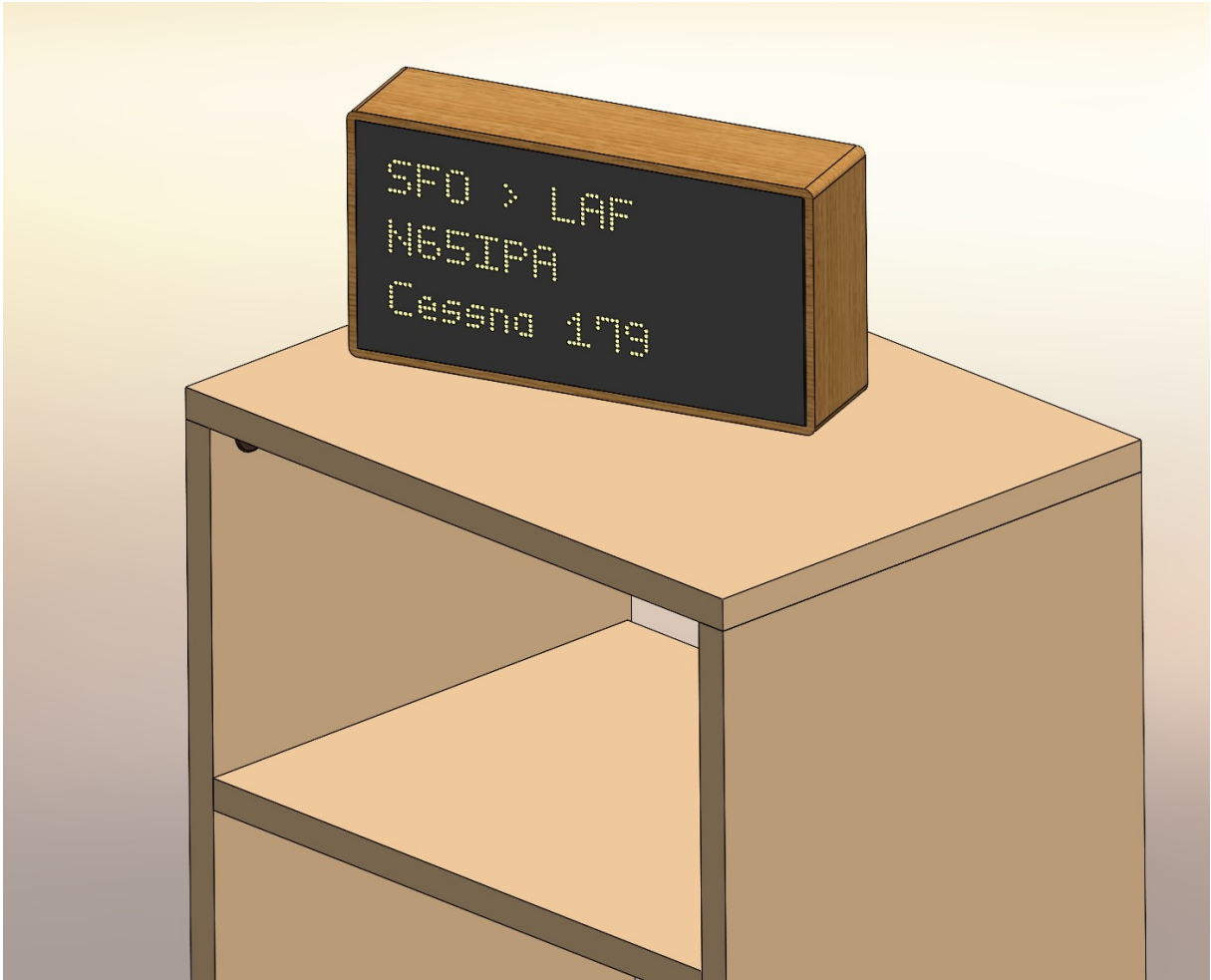


Figure 5.3.1: Aviator Enclosure on a generic Nightstand

5.4 Integration Approach

The system integrates an ADS-B software-defined radio (SDR), a cloud data pool, a local processing subsystem, a display module, and a power management subsystem. The ADS-B SDR publishes raw flight data to the cloud pool; the processing subsystem ingests that cloud stream plus external APIs and user inputs, normalizes and filters the data, and forwards compact display updates to the display module. The power management subsystem supplies energy to the processor and display and includes a battery backup to preserve state and reduce startup latency.

Design priorities are resilience and graceful degradation: if network connectivity is lost the device continues to show local time and the last cached flight/weather state; if power to the device is interrupted the ADS-B transmitter can still upload to the cloud independently. Data is cached locally and keyed by timestamp so that, on reconnection, the processor can quickly reconcile and display missed updates. Interfaces are defined around simple, versioned JSON payloads and REST/WebSocket endpoints to minimize coupling and simplify testing. Latency, cache expiration, and failure modes are documented per interface to guide integration testing.

5.5 System Photographs



Figure 5.5.1: Photograph of assembled system

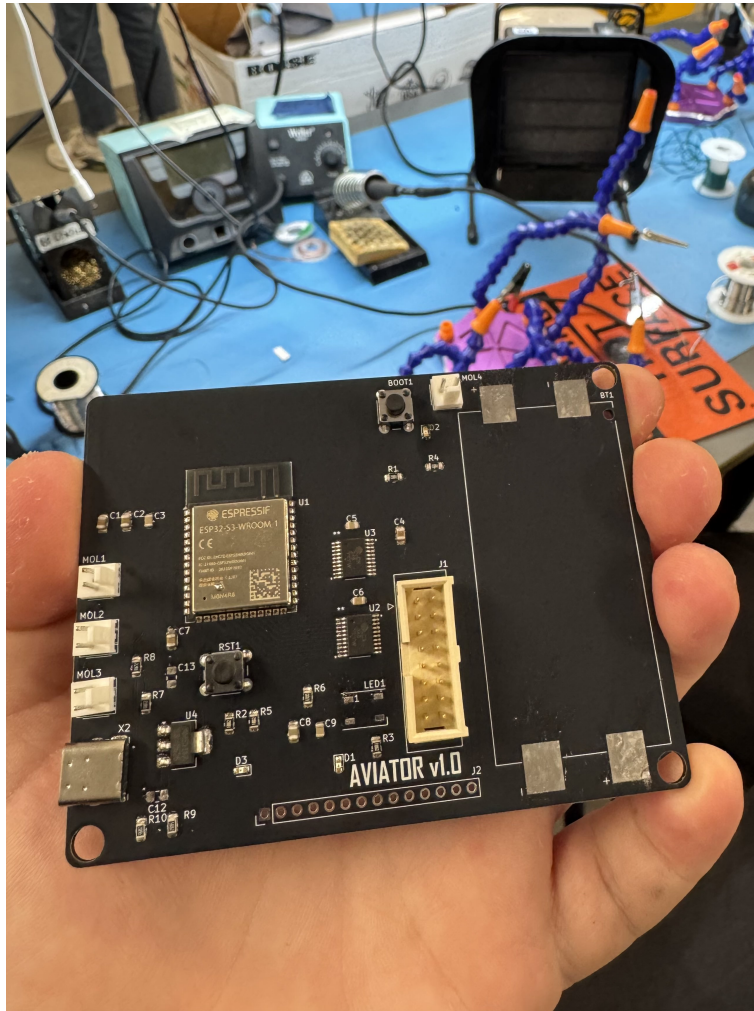


Figure 5.5.2: Photograph of assembled PCB

6 Subsystems

6.1 Subsystem 1: Processing

6.1.1 Subsystem Diagrams

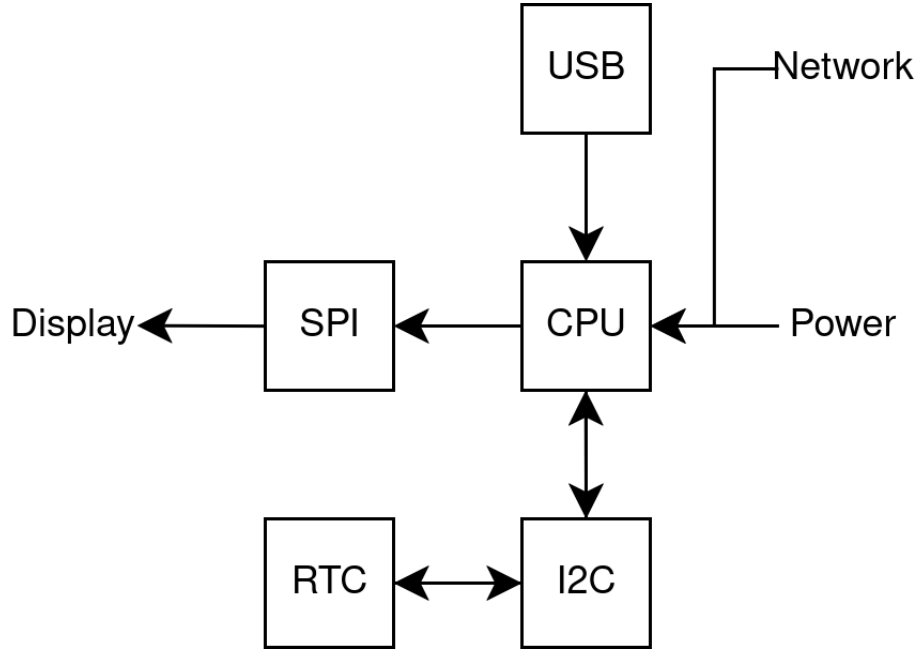


Figure 6.1.1: Processing subsystem Block Diagram

6.1.2 Specifications

1. Clock drift $\leq \pm 1 \frac{\text{sec}}{\text{day}}$
2. After network interrupt, system must reconnect within 120 seconds upon network resumption.
3. System must run with zero memory leaks.
4. System must never display text overrunning borders.

6.1.3 Subsystem Interactions

The core computer interfaces with all other subsystems. The battery/power management unit supplies it with power. Running processes direct and receive information from the network module. It communicates with the digital dot matrix display via SPI according to display drivers on the controller.

6.1.4 Core ECE Design Tasks

- **ENGR 16100:** Teamwork & project documentation.
- **CS 15900:** Fundamentals of programming.
- **ECE 36200:** PCB design and embedded software development.

6.1.5 Parts

- ESP32-S3
- DS3231
- PCB

6.1.6 Algorithm

```
initialize clock , network , display , location
DATA = {}

always
    wifi keep alive
    error handling
    if power button pressed
        save and shut down

every minute
    update display time

    read ADS-B sensor
    if ADS-B valid
        add to DATA
        if internet connected
            upload ADS-B data
        else
            cache ADS-B data
    else
        if internet connected
            make API call for flight data
            add to DATA
        else
            add warning ADS-B out of date to DATA
```

```

    if battery powered
        update battery level in DATA

    update time, weather in DATA
    convert DATA to pixel buffer
    push pixel buffer to display
every hour
    sync with network time
    make API call for weather

```

6.1.7 Theory of Operation

The processing subsystem is responsible for coordinating all other subsystems and serving as the “brain” of the Aviator device. At startup, the ESP32-S3 initializes its real-time clock, connects to the Wi-Fi network, and configures the SPI interface for display communication. During normal operation, the processor maintains a persistent network connection to ensure timely access to APIs.

Every minute, the processor updates the displayed time, retrieves the latest flight information from the network subsystem, and parses the response into a standardized data structure. If the device is battery-powered, it also queries the power management subsystem for voltage information and appends this to the displayed data. The processor then converts the compiled information (time, weather, and flight updates) into a pixel buffer and transmits it to the display subsystem.

On an hourly basis, the processor synchronizes the real-time clock with network time servers and fetches weather data for local context. Error handling routines ensure that loss of network, corrupted data, or power fluctuations do not crash the system; instead, the subsystem retries API calls or falls back to cached data, with a warning that the data is out of date.

6.1.8 Specifications Measurement

1. *Clock drift* $\leq \pm 1 \frac{\text{sec}}{\text{day}}$.

We measured drift over a period of two days with the following script:

```

import time, board, rtc, wifi, socketpool, adafruit_requests
pool = socketpool.SocketPool(wifi.radio)
requests = adafruit_requests.Session(pool, None)
r = requests.get("http://worldtimeapi.org/api/ip")
ref = r.json()['unixtime']

```

```

r.close()
dev = rtc.RTC().datetime
dev_epoch = int(time.mktime(dev[0:6] + (0,0,0)))
print("offset_s", dev_epoch - ref)

```

Reported drift was 1.44 seconds, or 0.72 seconds per day, within tolerance.

2. *After network interrupt, system must reconnect within 120 seconds upon network resumption.*

This was tested by disabling the Wi-Fi hotspot to which the device was connected, waiting 60 seconds, and reabling. Device reconnected in 50 seconds. By design, the device will always try to reconnect once a minute if disconnected.

3. *System must run with zero memory leaks.*

We tested this by enabling heap logging via the `gc` module and tracking over three days while device was in normal operation. Free memory was observed to not have a monotonically decreasing trend beyond noise; upon initialization, free memory was reported as 8274912 and after three days free memory was still reported as 8274912.

4. *System must never display text overrunning borders.* This is accomplished with bounds checking and scrolling text that overflows bounding box. Tested with random strings up to 65 characters long.

6.1.9 Standards

- **IPC-2221:** Governs PCB trace width, spacing, creepage/clearance, via rules, grounding, etc.
- **IPC-A-610:** Covers soldering quality and workmanship.
- **RFC 5905:** Protocol for syncing ESP time to internet.

6.2 Subsystem 2: Power Management

6.2.1 Subsystem Diagram

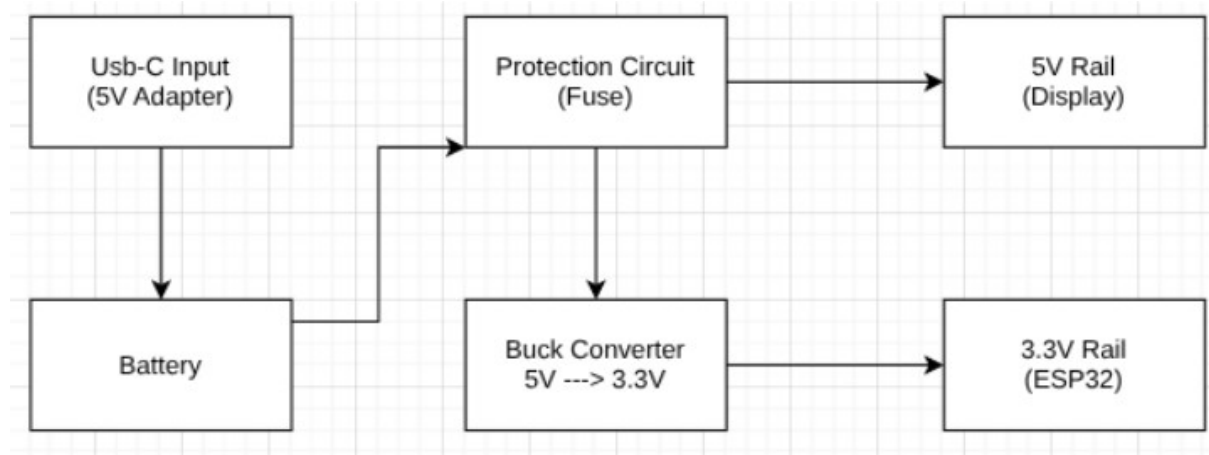


Figure 6.2.1: Power Management Subsystem Block Diagram

6.2.2 Specifications

1. Input Voltage: 5 V supplied through USB-C connector.
2. Backup Power Source: 3.7 V Li-ion battery (minimum 8000 mAh).
3. Output Rails:
 - Regulated 5 V rail for display subsystem (load up to 3 A).
 - Regulated 3.3 V rail for ESP32 and communications (load up to 1 A).
4. Voltage Accuracy: $\pm 5\%$ across all regulated rails.
5. Average System Power Consumption: ≤ 8 W under typical load.
6. Conversion Efficiency:
 - Buck/Buck-Boost conversion: $\eta = 85\%$.
 - Battery charge/discharge efficiency: $\eta = 80\%$.
7. Protection: Resettable polyfuse (3 A hold), TVS diode for surge protection.
8. Thermal Requirements: External device surface temperature ≤ 60 °C at full load.

6.2.3 Subsystem Interactions

- **Processing Subsystem:** Supplies a regulated 3.3 V rail to the ESP32, providing stable operation during Wi-Fi transmit peaks and computation bursts.
- **Display Subsystem:** Provides a high-current 5 V rail required for display backlight and brightness spikes.
- **Communications Subsystem:** Shares the same 3.3 V regulated supply for network operations and ADS-B retrieval.
- **PCB Integration:** Works with central board design to route power traces, place bulk and decoupling capacitors, and minimize noise on high-current paths.

6.2.4 Core ECE Design Tasks

- **ECE 25500 – Electronic Devices and Design Laboratory:**
Guides the design of buck and buck-boost stages, protection circuits, and voltage-regulation analysis.
- **ECE 27000 – Digital Systems:**
Interfaces battery status, “power-good” signals, and load switches with ESP32 logic.
- **ECE 36900 – Power Electronics:**
Provides theoretical basis for converters, load sharing, thermal considerations, and switching behavior.
- **ECE 49595 – Embedded Hardware Design:**
Applied in PCB layout, power routing, output filtering, and hardware integration.

6.2.5 Schematics

A detailed schematic is unnecessary since the power subsystem uses standard regulation and protection components, and the block diagram clearly shows the complete power flow and functionality.

6.2.6 Parts

- USB-C input controller: STUSB4500 (PD sink, 5 V mode)
- Buck-boost regulator: TI TPS63070 (3.3 V / 5 V output from battery or adapter)
- Battery charging IC: TP4056 (simple Li-ion charger for 5 V input)
- Li-ion battery pack: 3.7 V 8000 mAh flat pack with PCM protection

- Fuse: resettable polyfuse (3 A hold)
- TVS diode: SMBJ5.0A (for transient suppression)
- MOSFET switch: P-channel AO3407A (for reverse protection and load control)
- Connectors / Headers: keyed 5 V and 3.3 V output headers with test pads

6.2.7 Algorithm

1. Initialization: detect input source (USB-C present or battery only).
2. If USB-C is present: supply system load directly and charge battery at regulated current.
3. If USB-C is removed: automatically switch to battery through PowerPath MOSFET (ideal-diode behavior).
4. Monitor: ESP32 reads battery voltage and power-good signal periodically.
5. Low-voltage event: ESP32 alerts display and reduces brightness to conserve power.
6. Recharge: when USB-C is reconnected, return to normal operation and log battery status.

6.2.8 Theory of Operation

The Power Management Subsystem accepts 5 V from a USB-C adapter as its primary source. When external power is present, a charger IC distributes current to both the system load and the Li-ion battery. The buck-boost regulator maintains stable 3.3 V and 5 V rails regardless of whether the system is powered by the adapter or the battery.

If the USB-C connector is unplugged, a PowerPath MOSFET transitions the system to battery power with sub-100 ms interruption. The polyfuse and TVS diode protect against short circuits, surges, and reverse polarity conditions. The ESP32 monitors battery level and manages power-saving behaviors such as brightness reduction.

6.2.9 Specification Measurements

3.3 V Rail Accuracy (AMS1117-3.3 Test)

Test Setup: The AMS1117-3.3 linear regulator was powered from a 5 V USB supply. Output voltage was measured with a DMM referenced to system ground.

Results:

- Input Voltage: 5.0 V
- Target Output: 3.3 V

- Measured Output: 3.3018 V

Relative Error:

$$\text{Error} = \frac{3.3018 - 3.30}{3.30} \times 100\% = 0.052\%$$

This is well within the required $\pm 5\%$ specification. Therefore, the 3.3 V rail accuracy requirement is satisfied.

Additional tests (DD3 requirement):

- Battery charge current and termination level.
- Buck-boost conversion efficiency under 0.5 A, 1 A, and 2 A loads.
- Thermal measurement confirming external temperature $\leq 60^\circ\text{C}$ at full load.
- Failover time from USB $\rightarrow$ battery measured using oscilloscope (≤ 100 ms).

6.2.10 Standards

- USB-C Power Delivery Specification — governs USB-C negotiation and safe 5 V input.
- IEEE 1625 — battery reliability standard for portable electronics.
- UL 2054 — safety standard for rechargeable consumer batteries.
- IEC 62368-1 — safety for IT/communication equipment powering low-voltage systems.

The Power Management Subsystem meets all electrical and safety requirements for the Aviator System. Measurements confirm stable power delivery, reliable failover behavior, and safe thermal performance. No design changes were required for the final subsystem revision.

6.3 Subsystem 3: Text Display

6.3.1 Subsystem Diagrams

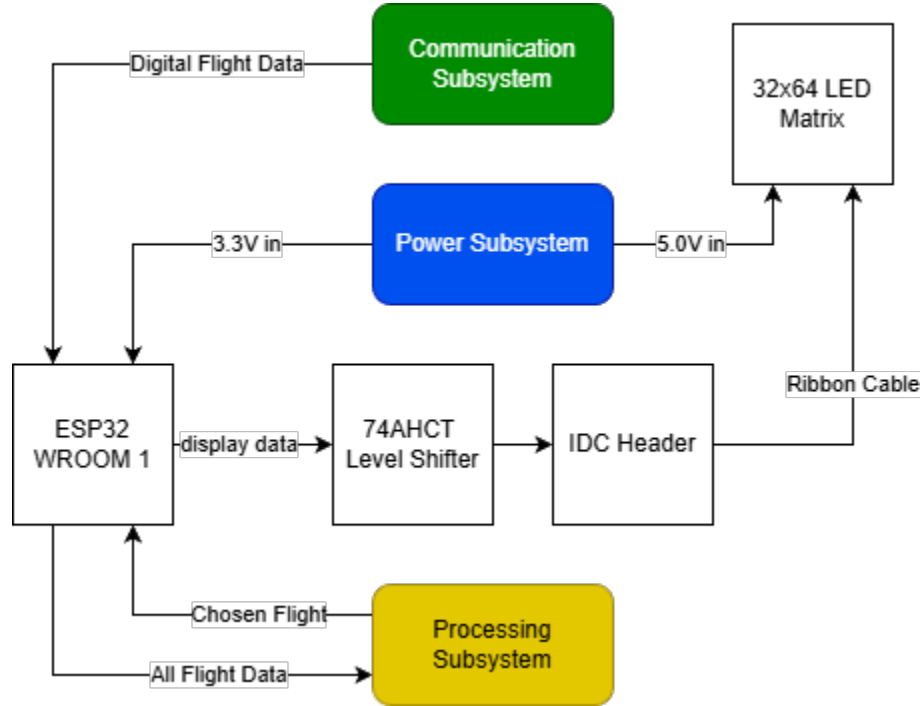


Figure 6.3.1: Display subsystem Block Diagram

6.3.2 Specifications

1. Feature a 2048-pixel Dot Display (64 x 32px)
2. Physical dimensions of housing within 5% of 64 x 128 x 256 mm.
3. No noticable screen tear (refresh rate $\geq$ 60Hz)

6.3.3 Subsystem Interactions

The display subsystem links the main PCB and the LED Matrix. This subsystem will interface all subsystems on and the main PCB. It will receive data from the network and communications subsystem, from the main PCB and processing subsystem it will receive commands regarding the format of the display, interpret the commands, and display the relevant air traffic data.

6.3.4 Core ECE Design Tasks

- **ECE 27000:** Provides a solid foundation in logic circuits. Helps for designing registers, data paths, logic that drives the LED panel.
- **ECE 25500:** Provides a base understanding of transistors, amplifiers, and fundamentals into I-V behavior.
- **ECE 40862:** Teaches valuable STM concepts such as programming, interrupts, and DMA which are needed to drive the display controller efficiently.

6.3.5 Schematics

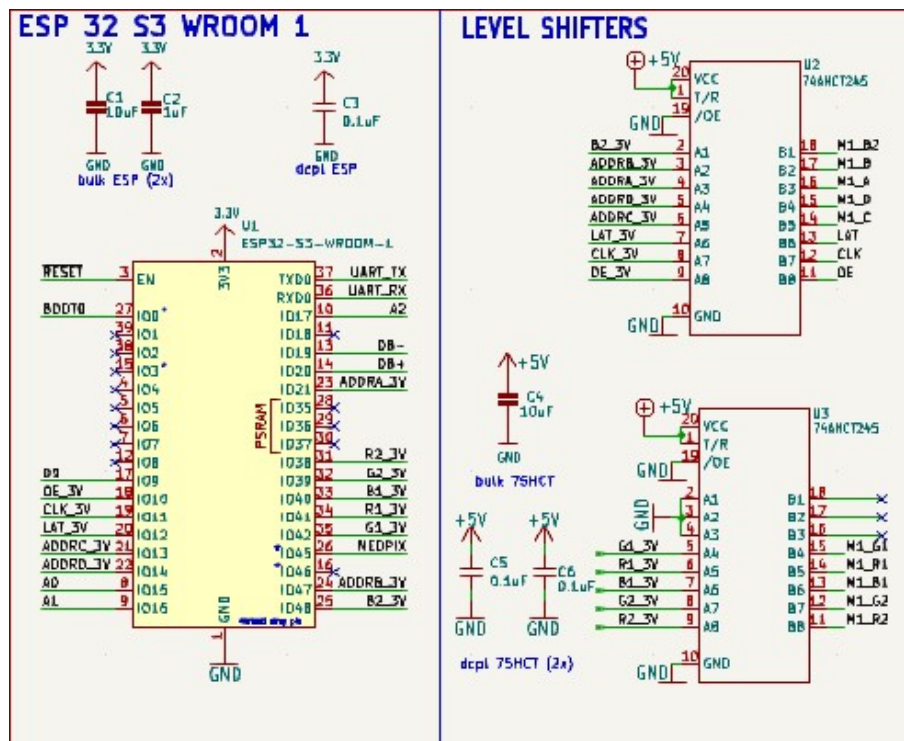


Figure 6.3.2: Display Driver Schematic (1/2)

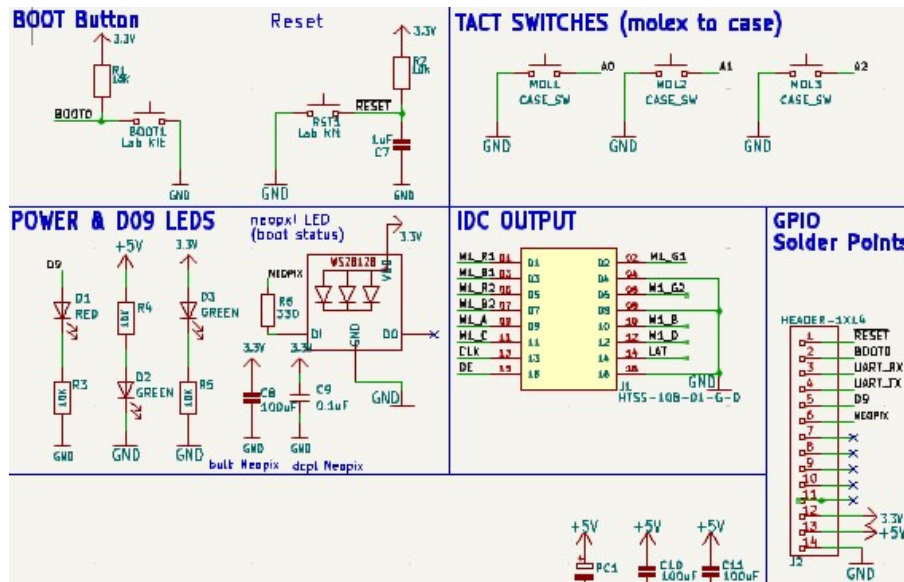


Figure 6.3.3: Display Driver Schematic (2/2) and features

6.3.6 Parts

- LED Matrix (ADAFruit, 32x64, P4)
 - 02x08 Pin header (IDC Connection)
- Resistors:
 - 330
 - 10k (5)
- Capacitors
 - 0.1uF (0603) (3)
 - 1uF (0805)
 - 10uF (0805)
 - 100uF (0805)
- Diodes
 - Red (0603)
 - Green (0603) (2)
- Switches and Buttons

- 01x02 Molex (Case Switch) (3)
- 6mm THM Push Button (2)

6.3.7 Algorithm

```

Initialize:
Configure GPIOs (DATA, CLK, LAT, OE, row select lines (A,B,C,D))
Initialize interface for incoming data
Initialize timer for refresh interrupts
Set PWM resolution (e.g., 8-bit)

Main Loop:
while (true):
    if new frame data available from main PCB:
        copy frame buffer into local memory (double buffer optional)

    for each row_index in 0 .. NUMROWS-1:
        select_row(row_index)
        OE = HIGH

        for pwm_bit = 7 downto 0:    // for 8-bit PWM
            for col_index = 0 .. NUMCOLS-1:
                pixel = frame_buffer[row_index][col_index]
                if pixel.red & (1 << pwm_bit) != 0:
                    DATA_R = HIGH
                else:
                    DATA_R = LOW
                if pixel.green & (1 << pwm_bit) != 0:
                    DATA_G = HIGH
                else:
                    DATA_G = LOW
                if pixel.blue & (1 << pwm_bit) != 0:
                    DATA_B = HIGH
                else:
                    DATA_B = LOW

            pulse(CLK)

        pulse(LAT)
        OE = LOW
        delay(PWMDELAY[pwm_bit])

```

repeat indefinitely

6.3.8 Theory of Operation

The Display subsystem serves as the primary interface between the ESP32-S3-WROOM-1 microcontroller and a HUB75d 32×64 RGB LED matrix, enabling real-time visualization of aviation and weather data. The ESP32 handles pixel generation, frame buffering, and timing control for row and column scanning of the display. Core signals — including LAT (latch), OE (output enable), CLK (clock), and address lines A–E — are driven directly from the ESP32's GPIO pins and level-shifted from 3.3 V logic to the 5 V domain required by the HUB75 interface. The RGB data lines (R1, G1, B1, R2, G2, B2) are similarly translated to ensure proper color channel switching for both the upper and lower halves of the LED matrix.

During operation, the ESP32 continuously cycles through the address lines, updating one row at a time while controlling brightness through pulse-width modulation synchronized with the OE signal. Power is supplied through regulated 5 V and 3.3 V rails, supporting both the LED panel and logic circuitry. Optional pushbuttons provide manual reset and boot functionality, while status LEDs indicate system power and data activity. In system deployment, the display subsystem retrieves aircraft position and weather telemetry from networked sources and ADS-B data, translating this incoming data into text output or images. This modular design allows the subsystem to function independently or as an integrated component of the larger Aviator platform, focusing on robustness, timing precision, and electrical isolation between logic and display domains.

6.3.9 Specifications Measurement

1. Feature a 2048-pixel Dot Display (64 x 32px)
2. The correct screen was ordered, featuring a 32x64px RGB LED matrix
3. Physical dimensions of housing within 5% of 64 x 128 x 256 mm.
4. Measured with ruler and calipers, the actual screen dimensions are 127 x 255 mm.
5. No noticable screen tear (refresh rate $\geq$ 60Hz)
6. In observing the screen during normal operation, the image is clear and constant with no noticable tear. Achieved $\geq$ 60Hz refresh rate

6.3.10 Standards

- **IEC 61010:** Safety for low-voltage electronic equipment; ensures protection against shorts, overcurrent, and handling risks.
- **SPI/I²C:** If MCU/controller communicates with peripheral ICs over standard buses.
- **HUB75:** Standard for 32×64 RGB LED matrices; timing, row multiplexing, and data latching must be followed.
- **JESD51:** Important for high-current LED arrays; ensures heat dissipation and junction temperatures are within safe limits.

6.4 Subsystem 3: Communications Subsystem

6.4.1 Subsystem Diagram

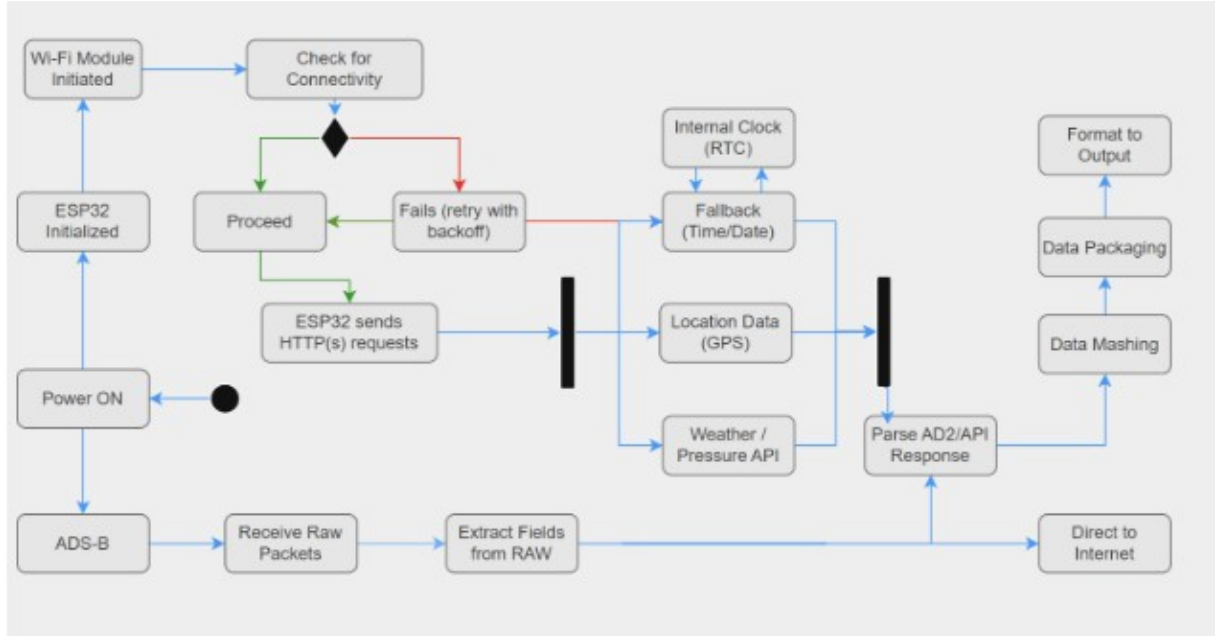


Figure 6.4.1: Communications Subsystem Block Diagram

6.4.2 Specifications

Wi-Fi & Internet Connectivity

1. Connection to WPA2 network established in less than 10 s.
2. HTTPS request latency less than 500 ms for payloads up to 2 kB.
3. Outbound HTTP POST/GET success rate at least 95%.
4. Fallback mode automatically activates within 5 s if connectivity is lost.
5. Error handling uses up to 3 retries with exponential back-off before fallback.
6. System connectivity uptime at least 95% during a continuous 1-hour runtime.

ADS-B Reception

1. Receives ADS-B messages over HTTPS from a remote decoding server.

2. Retrieval latency less than 500 ms per request.
3. API update frequency between 1–2 Hz (matching typical ADS-B refresh rate).
4. Data integrity ensured via JSON schema validation and CRC-validated ADS-B frames from the upstream server.
5. Missing-data tolerance: subsystem enters degraded-mode if $t = 5$ seconds pass without ADS-B updates.

GPS (Optional)

1. Provides 1 Hz location fix rate.
2. Supplies location and time data to synchronize ADS-B and API results.

6.4.3 Subsystem Interactions

The Communications Subsystem acts as the Aviator’s data bridge between external information sources and the internal electronics. It collects ADS-B data (via HTTPS), airport-information API responses, and weather API data, then forwards validated information to the Processing Subsystem over UART or SPI for formatting and display rendering.

Power is provided by the Power Management Subsystem through a regulated 3.3 V rail, ensuring stable operation of the ESP32 and associated communication hardware. The ESP32 can also report basic voltage or connectivity status back to the power subsystem as needed.

Externally, this subsystem interacts with:

- An online ADS-B server providing decoded aircraft position, velocity, and identification data.
- The Open-Meteo weather API.
- Airport-information APIs.

All ADS-B demodulation and decoding of raw I/Q samples is performed externally (e.g., on a host PC or remote server). The ESP32 retrieves only the decoded data frames through periodic HTTPS requests.

6.4.4 Core ECE Design Tasks

- **ECE 36200 – Microprocessor Systems and Interfacing:**
Implementing ESP32 programming, Wi-Fi configuration, SPI/UART communication, and coordinating real-time I/O with GPS and ADS-B modules.

- **ECE 43800 – Digital Signal Processing:**
Understanding ADS-B packet structure, CRC validation, and the signal-processing chain used during the external decoding process.
- **ECE 54700 – Computer Communication Networks:**
Applying network protocol knowledge (TCP/IP, congestion control, retry strategies, exponential backoff, and reliability mechanisms) to design the HTTPS communication, retry logic, and fallback behavior.
- **ECE 20875 – Python for Data Science:**
Implementing JSON parsing, data handling, and API integration for weather and ADS-B retrieval processes, including associated Python-side tooling.

6.4.5 Schematics

[Insert **DD2** schematic or pseudo-code diagram here.]

6.4.6 Parts

- **ADS-B Receiver (1090 MHz SDR Dongle):**
Receives aircraft broadcasts; raw I/Q data is decoded externally and uploaded to a server. ESP32 downloads decoded ADS-B messages over HTTPS.
- **Weather API Module:**
Software interface for retrieving real-time Open-Meteo weather data via HTTPS.
- **Wi-Fi Network Interface (ESP32 Integrated):**
Provides WPA2 networking and TCP/IP stack for ADS-B, weather, and airport-information retrieval.
- **Data/Signal Interface:**
Internal software paths for merging ADS-B, GPS, and API data and transmitting it to the Processing Subsystem.
- **GPS / Geolocation Unit (NEO-6M, optional):**
Provides 1 Hz location and timing data for correlating ADS-B positions and enhancing system accuracy.

6.4.7 Algorithm

This subsystem uses a multi-threaded structure that handles ADS-B retrieval and weather updates concurrently.

ADS-B Processing Pipeline

1. **Data Acquisition:** Retrieve decoded ADS-B messages from an external HTTPS server.
2. **Validation:** Apply JSON schema checks and ensure upstream CRC validation.
3. **Message Classification:** Use Type Codes (TC 1–4, 9–18, 19) to determine message type.
4. **Position Decoding:** Apply Compact Position Reporting (CPR) algorithms if raw coordinates require even/odd frame pairing.
5. **Database Management:** Maintain active aircraft records indexed by ICAO address.

Weather Integration

1. Issue HTTPS calls every 15 minutes to retrieve surface and upper-air data.
2. Parse fields including temperature, wind, and pressure at multiple atmospheric levels.
3. Correlate weather data to aircraft positions for contextual flight information.

Flight Database Correlation (Future)

1. Callsign lookup using OpenSky or FlightAware.
2. Route and delay correlation using external scheduling APIs.
3. Enhanced visualization of flight metadata.

Data Fusion and Export

1. Compute aircraft distance from receiver using geodesic formulas.
2. Apply retry handling for malformed frames and API failures.
3. Export fused data via high-level APIs to downstream processing.

6.4.8 Theory of Operation

On startup, the ESP32 connects to Wi-Fi and immediately begins polling an external ADS-B decoding server. Each HTTPS GET request retrieves a JSON payload containing aircraft ICAO addresses, positions, velocities, and timestamps.

Simultaneously, the ESP32 issues periodic HTTPS requests to the Open-Meteo API for weather conditions at surface and upper-atmosphere levels. If GPS is included, the module provides a synchronized 1 Hz fix for correlating ADS-B tracks with real-world position and time.

The subsystem runs continuously with:

- Failover behavior if connectivity is lost.
- Retry logic (three attempts with exponential backoff).
- Database pruning to remove stale aircraft entries.

6.4.9 Specification Measurements

Wi-Fi & Internet Connectivity

- Measure WPA2 connection time using UART timestamps (pass if < 10 s).
- Measure HTTPS latency using request/response timing (95th percentile < 500 ms).
- Compute success rate from 100–200 requests (pass if $\geq 95\%$).
- Disable Wi-Fi to test fallback activation time (< 5 s).
- Log retry intervals to confirm exponential backoff.

ADS-B Retrieval

- Measure GET request latency (95th percentile < 500 ms).
- Verify update frequency (1–2 Hz).
- Validate JSON schema; malformed packets $< 1\%$.

Weather API

- Verify call period is 900 ± 5 s.
- Ensure all expected JSON fields are present in 100% of responses.

Serial / SPI Output

- Measure throughput from the ESP32 to Processing Subsystem.
- Measure forwarding latency (pass if < 50 ms).

6.4.10 Standards

- **ADS-B (1090ES / 978 UAT)** — Ensures compatibility with standardized aircraft broadcast formats.
- **NMEA 0183** — GPS message formatting.
- **REST/HTTPS** — Secure communication with ADS-B and weather servers.
- **IEEE 802.11 / 802.3** — Networking standards for Wi-Fi/Ethernet.
- **TCP/IP & checksum validation** — Error detection and robust communication handling.



Figure 6.4.2: Communications Subsystem Schematic (Placeholder)

7 PCB Design

7.1 PCB Schematics

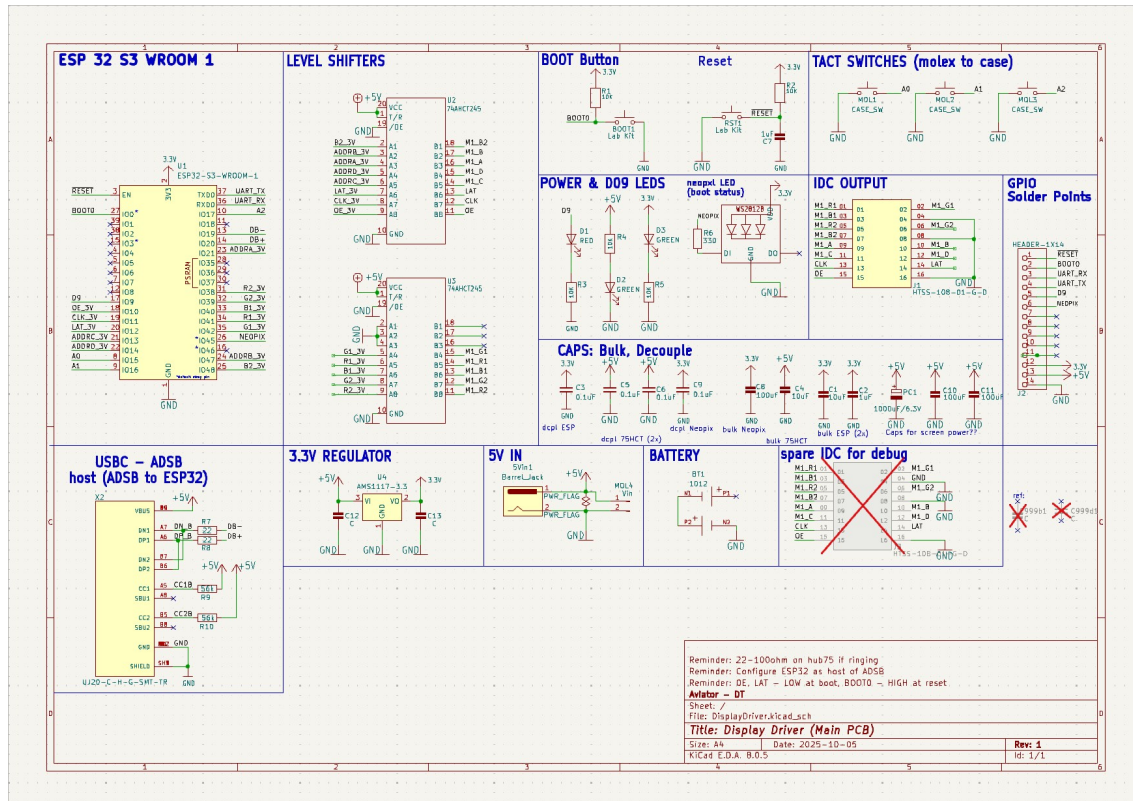


Figure 7.1.1: PCB Schematic

7.2 PCB Layout

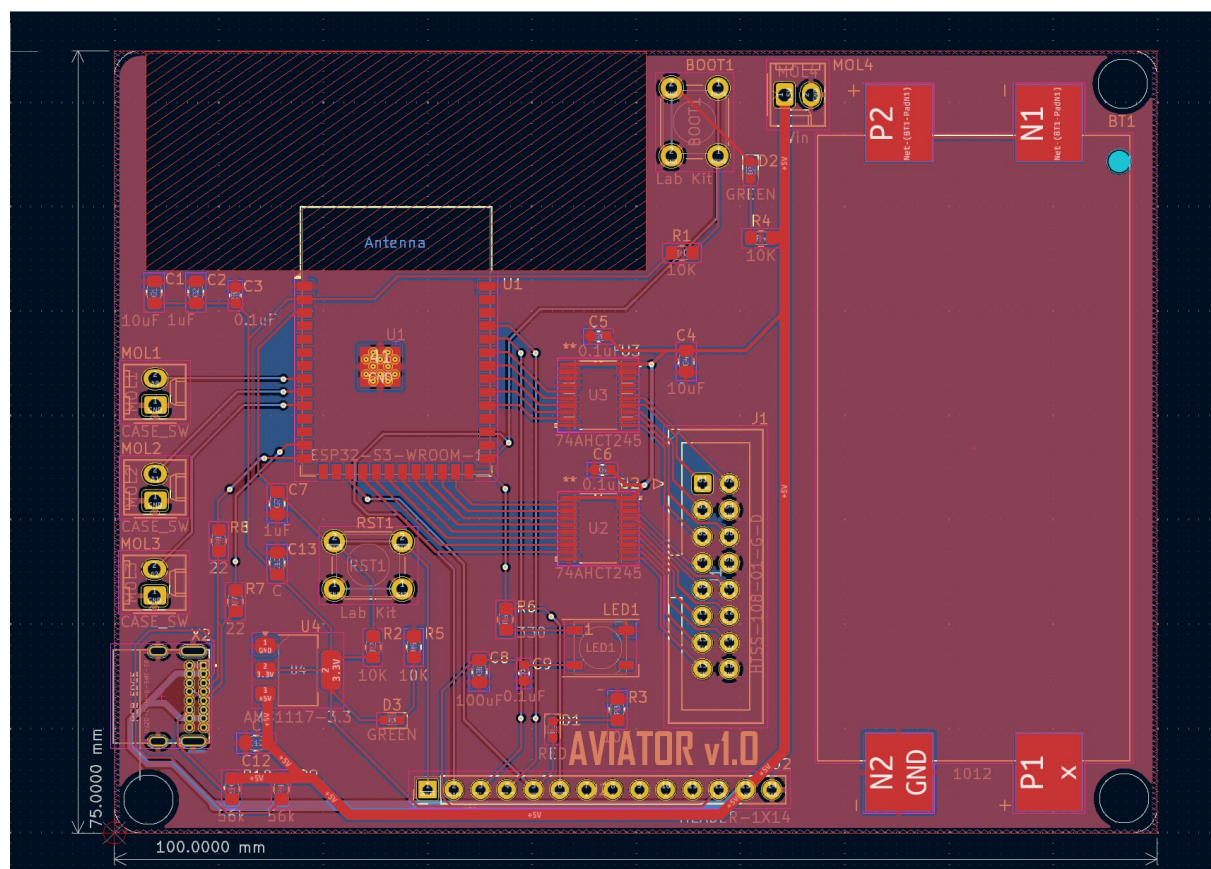


Figure 7.2.1: PCB Layout

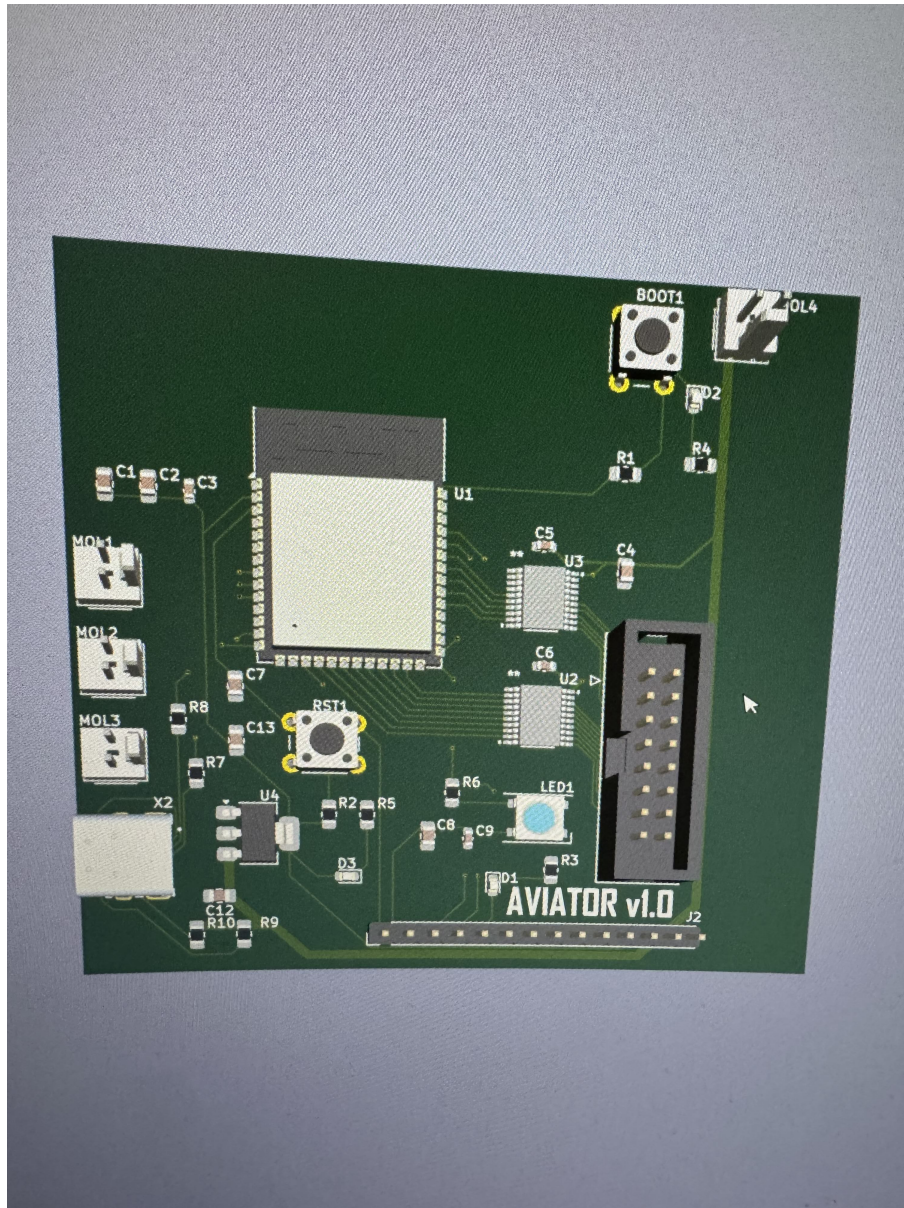


Figure 7.2.2: PCB CAD Layout

8 Final Status of Requirements

1. Requirement 1: The device must display accurate information.
Met: all information displayed is timestamped and discarded once stale. The device is paired with a local ADS-B receiver to ensure coverage of local flight since open flight APIs lack providers in most areas, ensuring overhead flights are accurately captured on-screen.
2. Requirement 2: The display must not interfere with user's well-being by, for example, displaying at excessive luminosity, updating rapidly in a distracting manner, or being excessively bulky.
Met: luminosity is configurable, animations are limited to scrolling overflowed text, and the device is contained within a 10x5x3 cubic inch volume.
3. Requirement 3: The device must not infringe on any person's reasonable expectation of privacy.
Partially met: the device's location could be determined by anyone with access to the API by plotting the epicenter of the newly appearing flights, and arguably tracking flights violates the privacy of flyers, but overall the device leaks very little personally identifiable information.
4. Requirement 4: The device must be language-agnostic wherever possible.
Not met: the interface is in English due to the difficulty of displaying discernible language-agnostic icons on a screen with 32x64 resolution. The documentation is also English.
5. Requirement 5: The device must be responsive and intuitive.
Met: the user interface is limited to swapping in between different display modes. All other configuration is automatic. The device responds in a fraction of a second to user input.
6. Requirement 6: The device must have robust error handling and recovery.
Met: the device is capable of autonomously recovering from any software error.
7. Requirement 7: The physical device should be easily replicated with widely available parts.
Met: by design, every component of the device is affordably sourced, with no custom machining needed beyond the PCB. A hobbyist or electrical engineering student should be able to recreate the device in a few hours.
8. Requirement 8: The code for the device must be open-source and well-documented.
Partially met: the code and PCB designs are published with open-source licenses. However, the documentation is unaligned with good practice and lacking in several areas.

9. Requirement 9: The device should be as durable and environmentally friendly as possible so as not to contribute to e-waste.
Partially met: the device is designed to last a minimum of four years of continuous operation, but the components are not sourced sustainably, and construction is not eco-friendly.
10. Requirement 10: The device must not contribute to noise or visual pollution of any space.
Met: the device emits no sound and displays no visually disrupting imagery.
11. Requirement 11: The device must be energy-efficient.
Partially met: the display consumes approximately 1.3 watts during normal operation, 16% of the maximum possible power consumption of the RGB LED display. While not low-power, the device therefore does use what power is available conservatively.
12. Requirement 12: The device must minimize construction and recurring costs.
Met: the device costs approximately \$110 USD to construct, including the ADS-B SDR. By using only free APIs, the device incurs no recurring costs apart from energy use.
13. Requirement 13: The device must not infringe on right to repair.
Met: all information needed to construct, maintain, and repair the device is freely available.
14. Requirement 14: The device must mount and dismount without damage to vertical surfaces.
Met: through magnets embedded in the casing, the device can attach to magnetic surfaces without adhesive. The back of the device is also conducive to non-damaging adhesive strips.

9 Team Structure

9.1 Team Member 1



David Thoe

Major: Electrical Engineering

Contact: dtthoe@purdue.edu

Team Role: Team Leader

Bio: In charge of graphics drivers and text display, I will be focused on the final presentation of the information as well as subsystem integration. I will also be paying special attention to the housing of the prototype, aided by CAD and 3D printing, ensuring the product appears polished when complete. At Purdue, I concentrate in Wireless and Optical engineering and participate in the club Autonomous Motorsports Purdue, where we place our fully autonomous go-kart in competition with other universities. In my free time, I work on hobby electrical projects usually related to my computer or decoration.

9.2 Team Member 2



Joshua Kim

Major: Electrical Engineering

Contact: kim3503@purdue.edu

Team Role: Communication Lead

Bio: In my role as Communication Lead, I am responsible for ensuring that communication on our team, with GTA, and the Professor is clear and consistent. I manage communications on updates, facilitate collaboration on progress reporting, and keep everyone engaged

and aligned within the subsystem groups to further our design as a cohesive project. My background in Electrical Engineering enables me to contribute directly to the technical aspects of the project and also help translate technical details into clear explanations for our team and other stakeholders. Balancing the communication and development of the subsystem, I can keep both the engineering work and project coordination rolling smoothly.

9.3 Team Member 3



Zeke Ulrich

Major: Computer Engineering

Contact: pulrich@purdue.edu

Team Role: Treasurer

Bio: Zeke is the processing and PCB design specialist. He's responsible for designing the PCB schematics and integrating the disparate subsystems on the processor. At Purdue, Zeke belongs to the Marine Corp Officer Candidate Program, Eta Kappa Nu, Tau Beta Pi, and Purdue's Effective Altruism community. Outside Purdue, he is president of the nonprofit DuelGood and works for the government in cybersecurity. In the future he hopes to study international relations as a Truman scholar, start a family, and volunteer as a firefighter. He enjoys athletics and spending time with his friends.

9.4 Team Member 4



Juan Vargas

Major: Electrical Engineering

Contact: varga105@purdue.edu

Team Role: Facilitator

Bio: As the Facilitator, my primary role is to keep our team organized, collaborative, and on track throughout the project. I focus on making sure discussions are productive, tasks are clearly divided, and deadlines are met without overwhelming any single member. By bridging technical conversations and helping resolve roadblocks quickly, I ensure that progress stays steady and balanced across subsystems. Alongside this coordination, I contribute to the technical development by assisting with software and system integration specially in the network/communication part, ensuring our device works as intended while maintaining the retro, aviation-inspired feel that defines Aviator.